SIMATS ENGINEERING
ASSESSMENT TOOL 1: Experiments

COURSE NAME: Cloud Computing and Big Data Analytics **COURSE CODE:** CSA1507

COURSE OUTCOME COVERED

CO3: *Implement cloud-based solutions using cloud storage, web APIs, and platform services for real-world applications.* (BL3)

Dave's Taxonomy: Precision (Level 3) **Total Marks: 40**
Question Count: 4

Code of Conduct

I _____JASMITHA R (192524295)_____ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.
Signature of the student: ___JASMITHA R____

Experiments Rubric

Criteria	Weightage	Level 4 - Exemplary	Level 3 - Proficient	Level 2 - Developing	Level 1 - Beginning
Experimental Design	25%	Design is systematic and technically strong	Mostly correct design	Basic design with gaps	Poorly designed activity
Use of Tools / Platforms	20%	Appropriate and effective use of cloud tools	Good tool usage with minor issues	Limited tool usage	Incorrect or missing tool usage
Application of Concepts	20%	Concepts are applied accurately to practical tasks	Mostly accurate application	Partial application	Weak application
Analysis and Output	20%	Strong analysis and meaningful outcome interpretation	Good analysis with minor gaps	Basic analysis	Little or no analysis
Documentation	15%	Clear, structured, and complete documentation	Mostly complete documentation	Partially complete	Poor documentation

Total Marks Awarded:

Experiments

Experiment 1: Create a simple cloud software application for Hospital information system for SIMATS Hospital using any Cloud Service Provider to demonstrate SaaS with fields of patient name, age, address, phone, email, Attender name, attender phone, doctor name, diseases, etc.

Experiment 2: Create a simple cloud software application for online super market using any Cloud Service Provider to demonstrate SaaS with the template of customer name, address, phone, email, items, quantity etc. with the item classification form.

Experiment 3: Demonstrate virtualization by Installing Type-2 Hypervisor in your device, create and configure VM image with a Host Operating system (Either Windows/Linux). Create a Virtual Machine with 3 CPU, 10GB RAM and 10GB storage disk using a Type 2 Virtualization Software.

Experiment 4: Demonstrate virtualization by Installing Type-2 Hypervisor in your device, create and configure VM image with a Host Operating system (Either Windows/Linux). Create a Snapshot of a VM and Test it by loading the Previous Version/Cloned VM.

Experiment 1: Hospital Information System (SaaS)

Aim

To design and deploy a simple cloud-based (SaaS) Hospital Information System for SIMATS Hospital using a Cloud Service Provider, enabling patient registration and record management such as patient details, attender details, doctor assigned, and diagnosed disease, accessible over the internet through a browser.

Software / Cloud Resources Required

- Cloud Service Provider account (AWS / Microsoft Azure / Google Cloud Platform — Free Tier)
- Static web hosting (Amazon S3 Static Website Hosting / Azure Static Web Apps / Firebase Hosting)
- Serverless backend (AWS Lambda + API Gateway, or Azure Functions, or Firebase Cloud Functions)
- Cloud database (Amazon DynamoDB / Azure Cosmos DB / Firebase Firestore)
- Web browser and a text editor (VS Code)

Database Fields — Patient Record Table

Field Name	Data Type / Description
Patient ID	String (Primary Key, auto-generated)
Patient Name	String
Age	Number
Address	String
Phone	String
Email	String
Attender Name	String
Attender Phone	String
Doctor Name	String
Disease / Diagnosis	String
Date of Admission	Date
Status	String — Admitted / Under Treatment / Discharged

Procedure

1. Create a free-tier account with the chosen Cloud Service Provider (this document uses AWS as an illustration).
2. Create a cloud database table named 'HospitalPatients' with 'PatientID' as the primary key, and add the attributes listed in the field table above.
3. Write a serverless function that exposes CRUD operations — Register Patient, Search Patient, Assign Doctor, and Update Status (Admitted/Under Treatment/Discharged).
4. Expose the function through API Gateway (or the equivalent HTTP trigger in Azure/GCP) to obtain a REST API endpoint.
5. Develop the front-end web page (HTML/CSS/JavaScript) shown below, which calls the REST API using fetch().

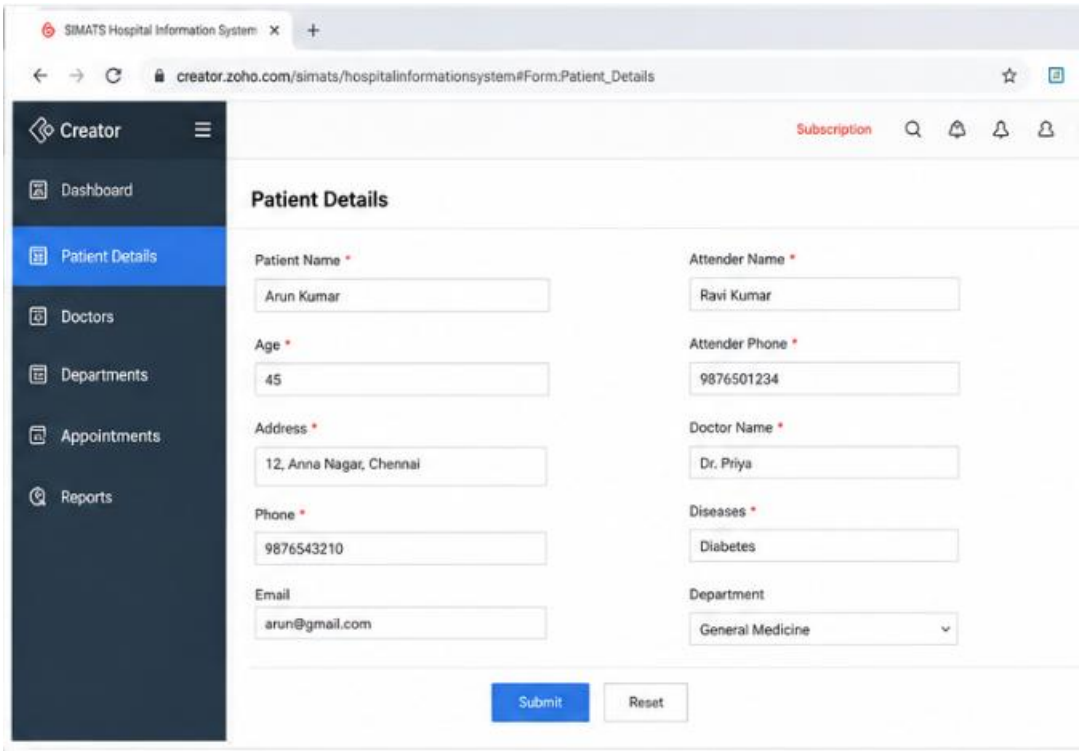
6. Upload the front-end files to an S3 bucket and enable Static Website Hosting (or deploy to Azure Static Web Apps / Firebase Hosting).
7. Set the bucket policy / hosting permissions to allow public read access so the application is reachable as a SaaS application over a URL.
8. Open the hosted URL in a browser, register sample patient records, and verify that doctor assignment and status updates work and reflect correctly in the cloud database.
9. This confirms the SaaS delivery model — the hospital information system runs entirely on the cloud provider's infrastructure and is accessed only through a browser, with no local software installation.

Sample Front-End Code (index.html)

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <title>SIMATS Hospital Information System</title>
  <style>
    body { font-family: Arial, sans-serif; margin: 30px; background:#f4f6f8; }
    h2 { color:#1f4e78; }
    table { border-collapse: collapse; width:100%; background:#fff; }
    th, td { border:1px solid #ccc; padding:8px; text-align:left; }
    th { background:#1f4e78; color:#fff; }
    input, select { padding:6px; margin:4px; }
    button { padding:6px 14px; background:#1f4e78; color:#fff; border:none; cursor:pointer; }
  </style>
</head>
<body>
  <h2>SIMATS Hospital - Patient Registration</h2>
  <div>
    <input id="pname" placeholder="Patient Name">
    <input id="age" placeholder="Age" type="number">
    <input id="address" placeholder="Address">
    <input id="phone" placeholder="Phone">
    <input id="email" placeholder="Email">
    <input id="attName" placeholder="Attender Name">
    <input id="attPhone" placeholder="Attender Phone">
    <input id="doctor" placeholder="Doctor Name">
    <input id="disease" placeholder="Disease">
    <button onclick="addPatient()">Register Patient</button>
  </div>
  <table id="patientTable">
    <tr><th>Name</th><th>Age</th><th>Phone</th><th>Attender</th>
      <th>Doctor</th><th>Disease</th><th>Status</th><th>Action</th></tr>
  </table>
  <script>
    const API = "https://<your-api-id>.execute-api.<region>.amazonaws.com/patients";

    async function loadPatients() {
      const res = await fetch(API);
      const patients = await res.json();
      const table = document.getElementById("patientTable");
      table.innerHTML = table.rows[0].outerHTML;
      patients.forEach(pt => {
        const row = table.insertRow();
        row.innerHTML = `<td>${pt.name}</td><td>${pt.age}</td><td>${pt.phone}</td>
          <td>${pt.attName}</td><td>${pt.doctor}</td><td>${pt.disease}</td>
          <td>${pt.status}</td>
```

```
      <td><button onclick="updateStatus('${pt.patientId}')">Update Status</button></td>`;
    });
  }

  async function addPatient() {
    const patient = {
      name: pname.value, age: age.value, address: address.value,
      phone: phone.value, email: email.value, attName: attName.value,
      attPhone: attPhone.value, doctor: doctor.value, disease: disease.value,
      status: "Admitted"
    };
    await fetch(API, { method: "POST", body: JSON.stringify(patient) });
    loadPatients();
  }

  async function updateStatus(id) {
    await fetch(`${API}/${id}`, { method: "PUT", body: JSON.stringify({ status: "Discharged" }) });
    loadPatients();
  }

  loadPatients();
 </script>
</body>
</html>
```

Output Screenshot



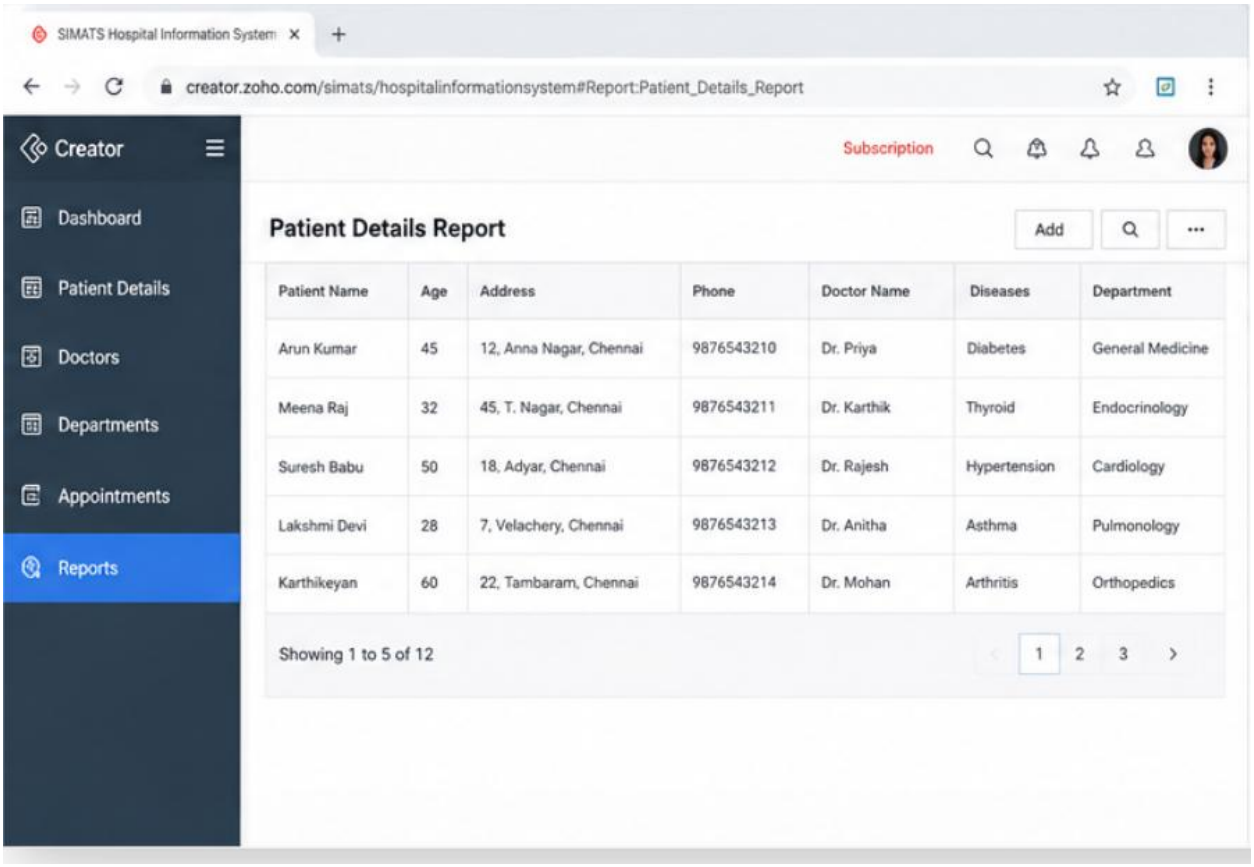


Fig 1.1 — Patient records output on the deployed SaaS application (Zoho Creator)

Result

A cloud-hosted Hospital Information System for SIMATS Hospital was successfully created and deployed as a SaaS application. The application allows registering patient records with attender and doctor details, and updating patient status (Admitted/Under Treatment/Discharged), all accessed through a public URL without any local installation.

Experiment 2: Online Super Market System (SaaS)

Aim

To design and deploy a simple cloud-based (SaaS) Online Super Market application using a Cloud Service Provider, allowing customers to register their details, browse item categories, and place orders with quantities, accessible over the internet through a browser.

Software / Cloud Resources Required

- Cloud Service Provider account (AWS / Azure / GCP — Free Tier)
- Static web hosting (Amazon S3 / Azure Static Web Apps / Firebase Hosting)
- Serverless backend for order processing (AWS Lambda / Azure Functions / Firebase Cloud Functions)
- Cloud database to store customer and order/item records (DynamoDB / Cosmos DB / Firestore)

Database Fields — Customer Order Table

Field Name	Data Type / Description
Order ID	String (Primary Key, auto-generated)
Customer Name	String
Address	String
Phone	String
Email	String
Item Name	String
Quantity	Number
Order Date	Date

Item Classification Form — Fields

Field Name	Data Type / Description
Item Code	String (Primary Key)
Item Name	String
Category	String — Grocery / Dairy / Vegetables / Fruits / Household / Beverages
Unit Price	Number (₹)
Unit of Measure	String — kg / litre / piece / packet
Stock Available	Number

Procedure

1. Create a free-tier account with the chosen Cloud Service Provider.
2. Create two cloud database tables: 'Items' (item classification, using Item Code as primary key) and 'Orders' (customer orders, using Order ID as primary key) with the attributes listed above.
3. Write a serverless function that allows adding items under a category (item classification form), listing items by category, and placing a customer order that reduces stock accordingly.

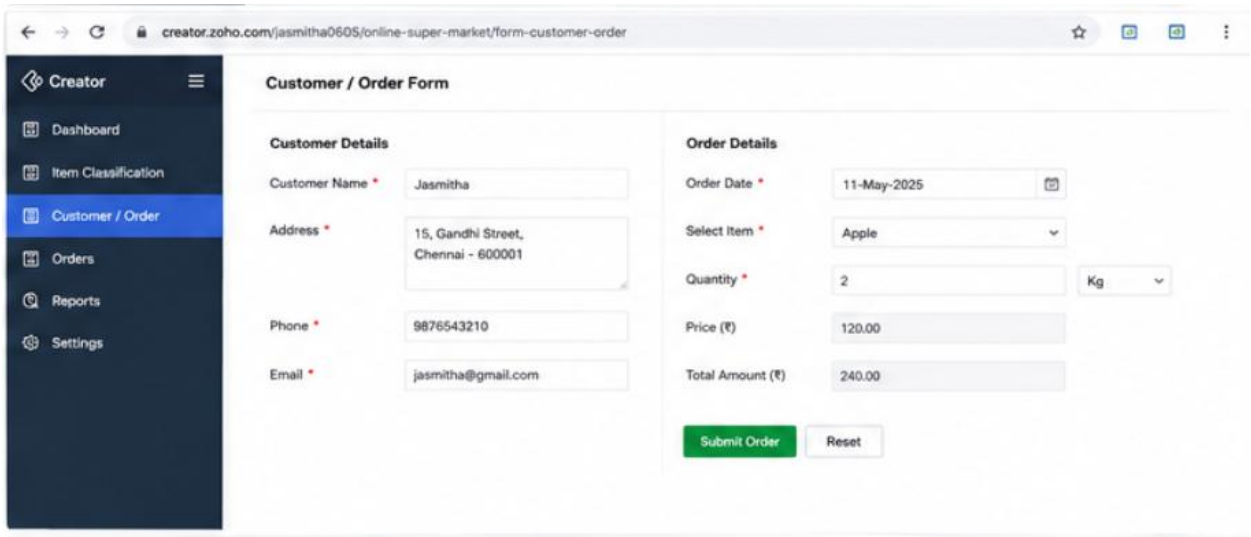
4. Expose the function as REST API endpoints using API Gateway (or equivalent) — one endpoint for items, one for orders.
5. Build the front-end web page shown below with two sections: an Item Classification form for the store admin, and a Customer Order form for shoppers.
6. Deploy the front-end to S3 Static Website Hosting (or Azure/Firebase Hosting) and enable public access.
7. Access the deployed URL, add sample items under different categories, then place a test customer order and verify that customer details, item, and quantity are stored correctly in the cloud database.
8. This confirms the SaaS delivery model — the super market application and its data are entirely cloud-hosted and reachable only through a browser.

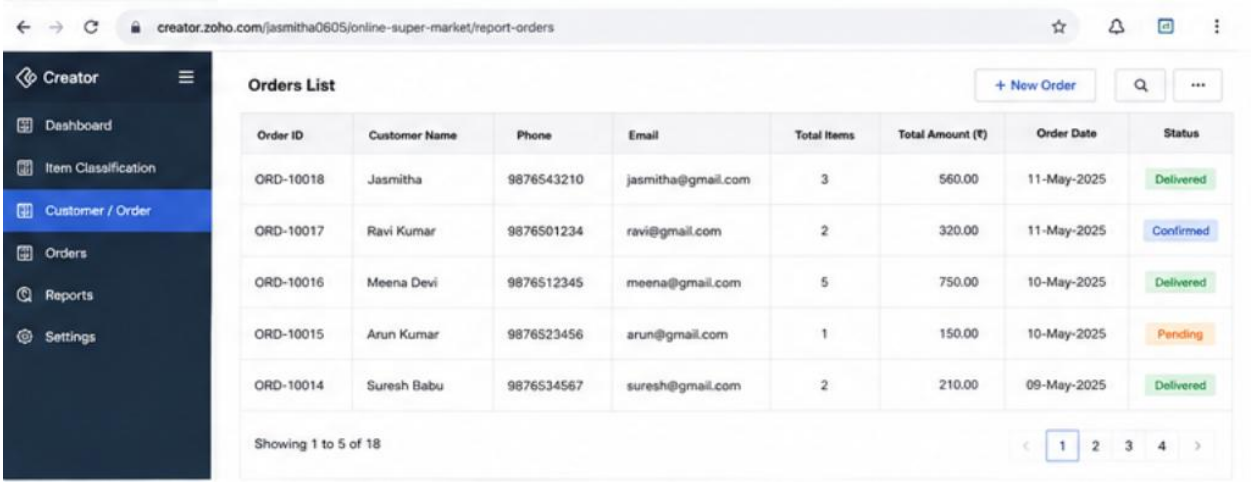
Sample Front-End Code (market.html)

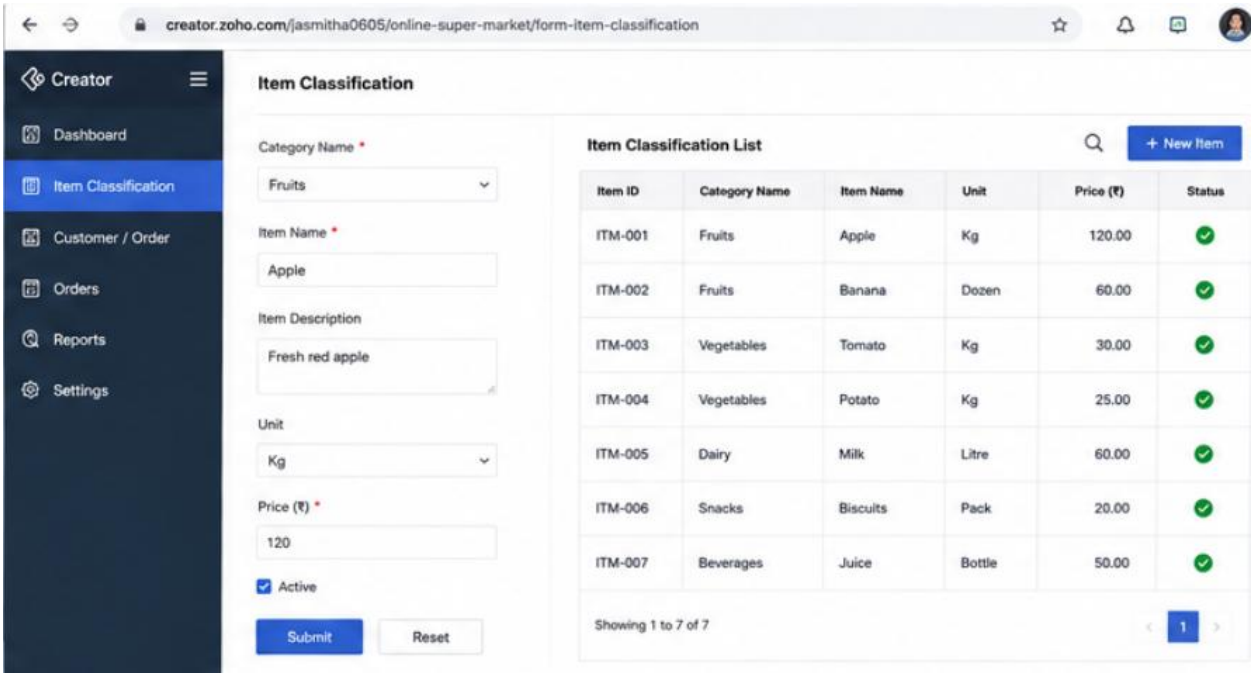
```
<!DOCTYPE html>
<html lang="en">
<head>
<meta charset="UTF-8">
<title>Online Super Market</title>
<style>
  body { font-family: Arial, sans-serif; margin:30px; background:#f4f6f8; }
  h2, h3 { color:#1f4e78; }
  .card { background:#fff; padding:20px; border-radius:6px; max-width:520px; margin-bottom:20px; }
  label { display:block; margin-top:10px; font-weight:bold; }
  input, select { width:100%; padding:6px; box-sizing:border-box; }
  button { margin-top:16px; padding:8px 16px; background:#1f4e78; color:#fff; border:none; }
</style>
</head>
<body>
  <h2>Online Super Market</h2>

  <div class="card">
    <h3>Item Classification Form</h3>
    <label>Item Code</label><input id="itemCode">
    <label>Item Name</label><input id="itemName">
    <label>Category</label>
    <select id="category">
      <option>Grocery</option><option>Dairy</option><option>Vegetables</option>
      <option>Fruits</option><option>Household</option><option>Beverages</option>
    </select>
    <label>Unit Price (₹)</label><input id="price" type="number">
    <label>Unit of Measure</label><input id="unit" placeholder="kg / litre / piece">
    <label>Stock Available</label><input id="stock" type="number">
    <button onclick="addItem()">Add Item</button>
  </div>

  <div class="card">
    <h3>Customer Order Form</h3>
    <label>Customer Name</label><input id="custName">
    <label>Address</label><input id="address">
    <label>Phone</label><input id="phone">
    <label>Email</label><input id="email">
    <label>Item Name</label><input id="orderItem">
    <label>Quantity</label><input id="qty" type="number">
    <button onclick="placeOrder()">Place Order</button>
  </div>
```

```
 <script>
  const ITEM_API = "https://<your-api-id>.execute-api.<region>.amazonaws.com/items";
  const ORDER_API = "https://<your-api-id>.execute-api.<region>.amazonaws.com/orders";

  async function addItem() {
   const item = {
    itemCode: itemCode.value, itemName: itemName.value,
    category: category.value, price: parseFloat(price.value),
    unit: unit.value, stock: parseInt(stock.value)
   };
   await fetch(ITEM_API, { method: "POST", body: JSON.stringify(item) });
   alert("Item added under category: " + item.category);
  }

  async function placeOrder() {
   const order = {
    custName: custName.value, address: address.value, phone: phone.value,
    email: email.value, itemName: orderItem.value, quantity: parseInt(qty.value)
   };
   await fetch(ORDER_API, { method: "POST", body: JSON.stringify(order) });
   alert("Order placed successfully!");
  }
 </script>
</body>
</html>
```

Output Screenshot




creator.zoho.com/jasmitha0605/online-super-market/report-orders
Creator
Dashboard
Item Classification
Customer / Order
Orders
Reports
Settings
Orders List
+ New Order
Order ID | Customer Name | Phone | Email | Total Items | Total Amount (₹) | Order Date | Status
ORD-10018 | Jasmitha | 9876543210 | jasmitha@gmail.com | 3 | 560.00 | 11-May-2025 | Delivered
ORD-10017 | Ravi Kumar | 9876501234 | ravi@gmail.com | 2 | 320.00 | 11-May-2025 | Confirmed
ORD-10016 | Meena Devi | 9876512345 | meena@gmail.com | 5 | 750.00 | 10-May-2025 | Delivered
ORD-10015 | Arun Kumar | 9876523456 | arun@gmail.com | 1 | 150.00 | 10-May-2025 | Pending
ORD-10014 | Suresh Babu | 9876534567 | suresh@gmail.com | 2 | 210.00 | 09-May-2025 | Delivered
Showing 1 to 5 of 18
1 2 3 4


creator.zoho.com/jasmitha0605/online-super-market/form-item-classification
Creator
Dashboard
Item Classification
Customer / Order
Orders
Reports
Settings
Item Classification
Category Name *
Fruits
Item Name *
Apple
Item Description
Fresh red apple
Unit
Kg
Price (₹) *
120
Active
Submit
Reset
Item Classification List
+ New Item
Item ID | Category Name | Item Name | Unit | Price (₹) | Status
ITM-001 | Fruits | Apple | Kg | 120.00
ITM-002 | Fruits | Banana | Dozen | 60.00
ITM-003 | Vegetables | Tomato | Kg | 30.00
ITM-004 | Vegetables | Potato | Kg | 25.00
ITM-005 | Dairy | Milk | Litre | 60.00
ITM-006 | Snacks | Biscuits | Pack | 20.00
ITM-007 | Beverages | Juice | Bottle | 50.00
Showing 1 to 7 of 7
1

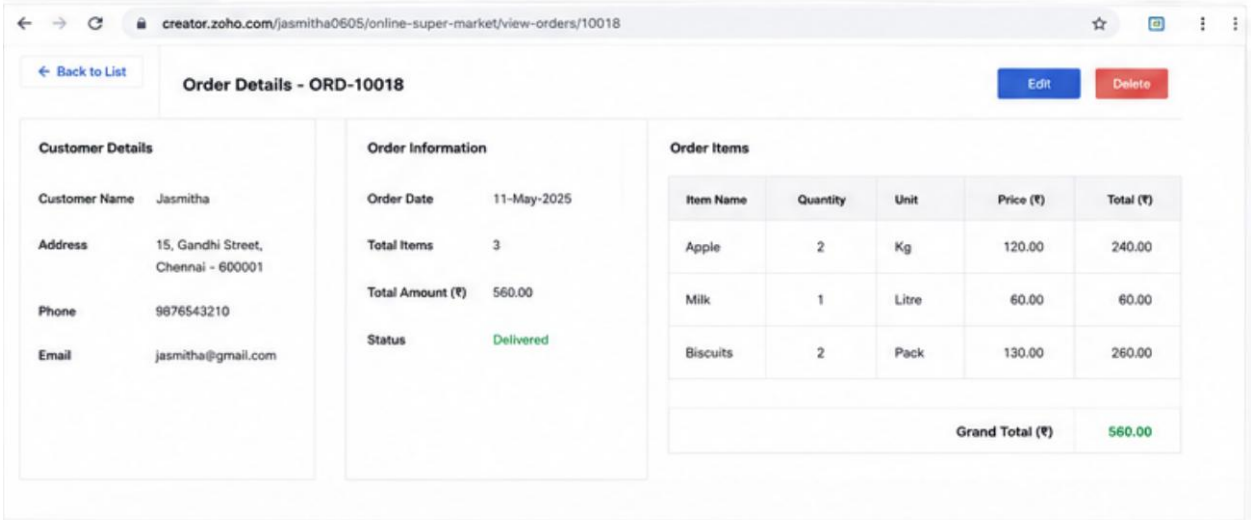

creator.zoho.com/jasmitha0605/online-super-market/view-orders/10018
← Back to List
Order Details - ORD-10018
Edit
Delete
Customer Details
Customer Name Jasmitha
Address 15, Gandhi Street, Chennai - 600001
Phone 9876543210
Email jasmitha@gmail.com
Order Information
Order Date 11-May-2025
Total Items 3
Total Amount (₹) 560.00
Status Delivered
Order Items
Item Name | Quantity | Unit | Price (₹) | Total (₹)
Apple | 2 | Kg | 120.00 | 240.00
Milk | 1 | Litre | 60.00 | 60.00
Biscuits | 2 | Pack | 130.00 | 260.00
Grand Total (₹) 560.00

Fig 2.1 — Item classification and customer order output on the deployed SaaS application (Zoho Creator)

Result

A cloud-based Online Super Market application was successfully created and deployed as a SaaS application. Items could be added and classified by category through the Item Classification form, and customers could place orders with item and quantity details, all stored in a cloud database and accessed through a public URL without local installation.

Experiment 3: Creating a Virtual Machine using a Type-2 Hypervisor

Aim

To demonstrate virtualization by installing a Type-2 Hypervisor on the host machine and creating a Virtual Machine with 3 vCPUs, 10 GB RAM, and a 10 GB storage disk, configured with a guest operating system (Windows/Linux).

Software Required

- Host machine with virtualization (Intel VT-x / AMD-V) enabled in BIOS/UEFI, and at least 4 physical/logical CPU cores and 12 GB+ RAM available
- Type-2 Hypervisor: Oracle VirtualBox 7.x (or VMware Workstation Player)
- Guest OS ISO image (e.g., Ubuntu Desktop/Server ISO, or Windows 10/11 ISO)
- At least 12 GB free disk space on the host

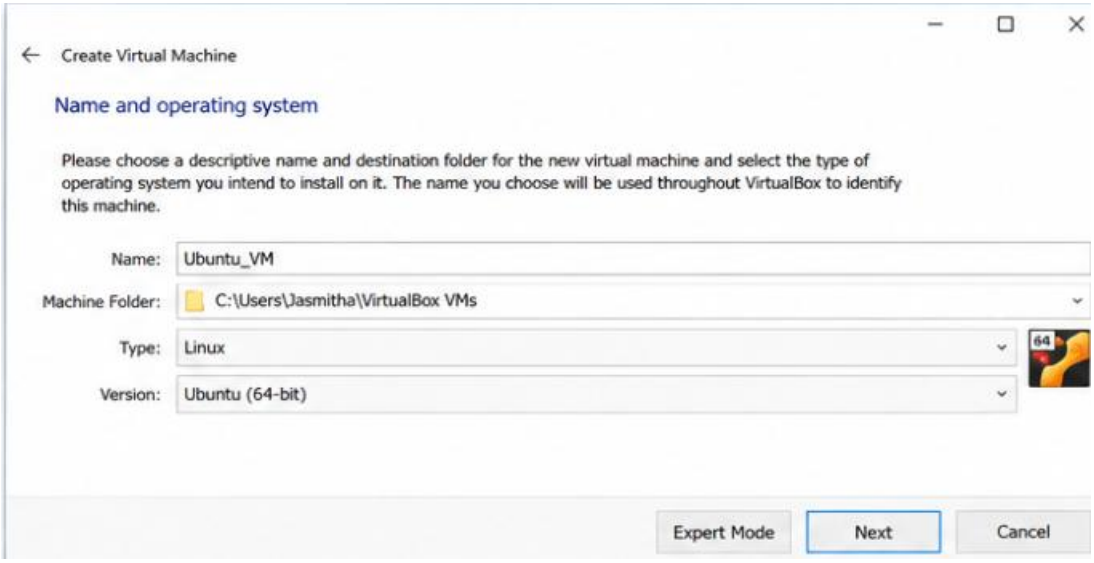
Procedure

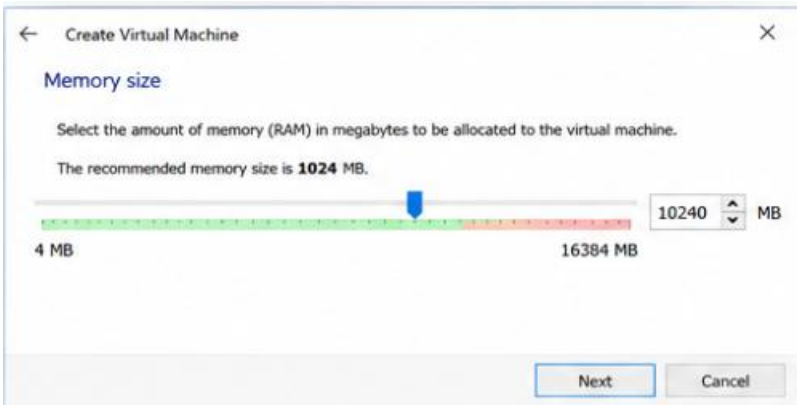
1. Enable virtualization support (Intel VT-x/AMD-V) in the host system's BIOS/UEFI settings.
2. Download and install Oracle VirtualBox (Type-2 Hypervisor) along with the VirtualBox Extension Pack on the host operating system.
3. Download the desired guest OS ISO image (e.g., Ubuntu Desktop) and save it on the host machine.
4. Open VirtualBox Manager and click 'New' to create a new virtual machine.
5. Enter a name for the VM (e.g., 'SIMATS-VM2'), select the Type (Linux/Windows) and Version matching the ISO chosen.
6. Set the Memory Size to 10240 MB (10 GB) as required, ensuring the host has enough free RAM remaining for itself.
7. Choose 'Create a Virtual Hard Disk Now', select VDI format, Dynamically Allocated, and set the size to 10 GB.
8. Select the newly created VM, go to Settings → System → Processor, and set the number of CPUs to 3.
9. Go to Settings → Storage, attach the downloaded ISO file to the virtual optical drive.
10. Start the VM; the guest OS installer boots from the attached ISO. Follow the on-screen steps to install the guest operating system onto the 10 GB virtual disk.
11. After installation completes, remove the ISO from the virtual drive and reboot the VM to boot from the virtual hard disk directly.
12. Verify the configuration from within the guest OS (System Info) confirming 3 CPUs, 10 GB RAM, and 10 GB disk, and install VirtualBox Guest Additions for better performance/integration.

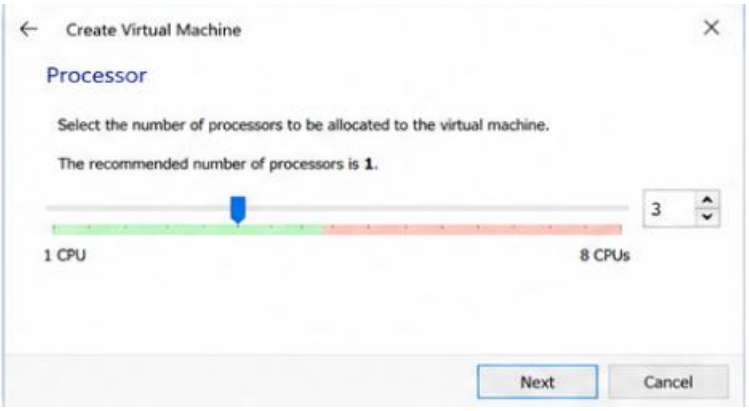
Configuration Summary

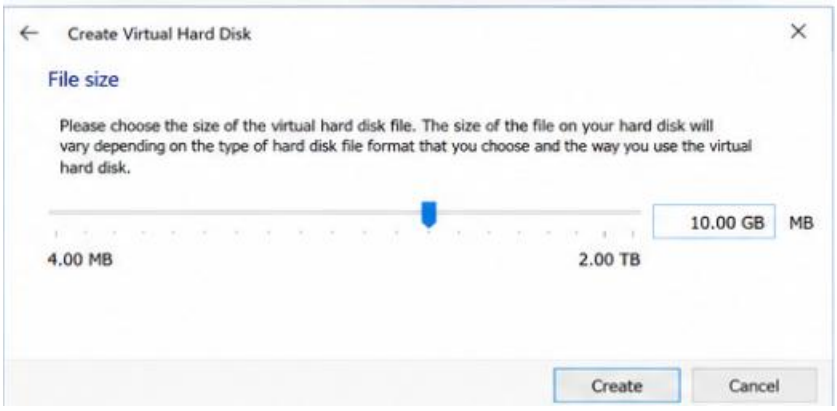
Parameter	Configured Value
Hypervisor Type	Type-2 (Hosted) — Oracle VirtualBox
Host OS	Windows / Linux (as installed on physical machine)
Guest OS	Ubuntu Linux / Windows (as selected)
vCPU	3
RAM	10 GB (10240 MB)
Storage Disk	10 GB (VDI, Dynamically Allocated)

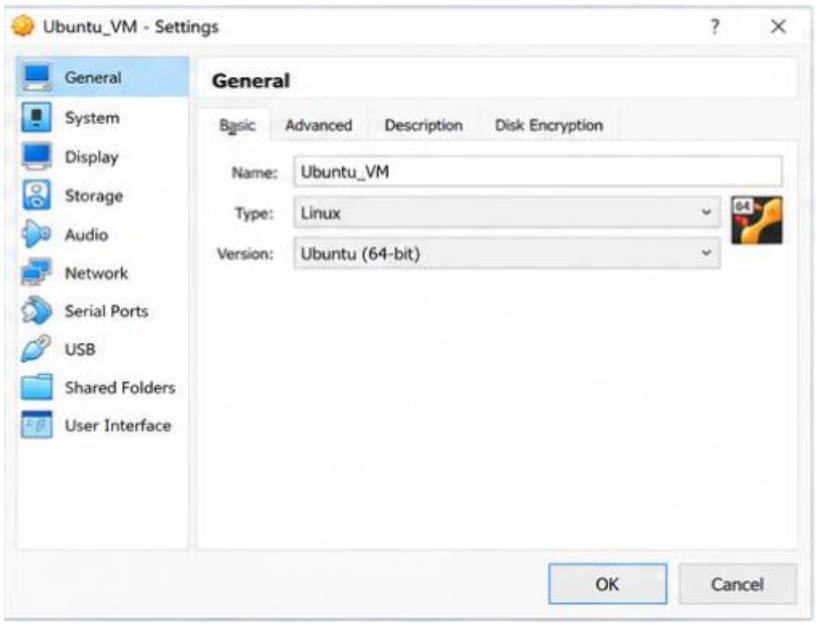
Output Screenshot

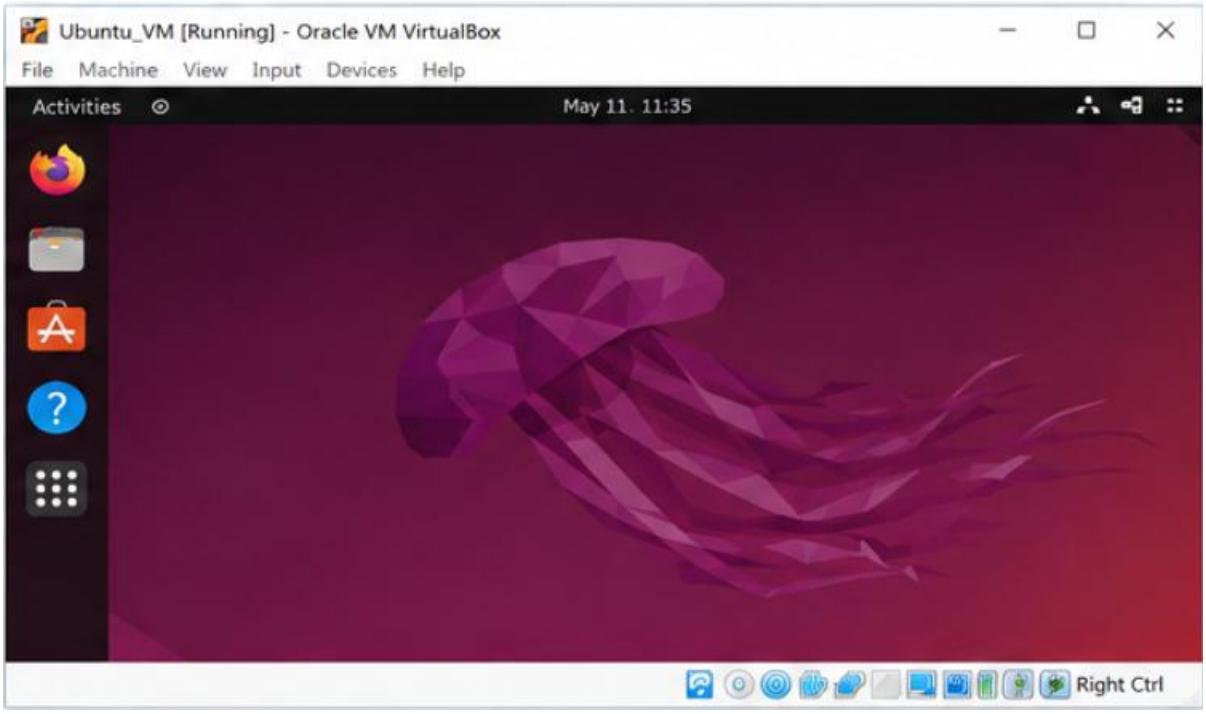





Create Virtual Machine
Processor
Select the number of processors to be allocated to the virtual machine.
The recommended number of processors is 1.
3
1 CPU
8 CPUs
Next
Cancel


Create Virtual Hard Disk
File size
Please choose the size of the virtual hard disk file. The size of the file on your hard disk will vary depending on the type of hard disk file format that you choose and the way you use the virtual hard disk.
10.00 GB
MB
4.00 MB
2.00 TB
Create
Cancel


Ubuntu_VM - Settings
General
System
Display
Storage
Audio
Network
Serial Ports
USB
Shared Folders
User Interface
General
Basic
Advanced
Description
Disk Encryption
Name:
Ubuntu_VM
Type:
Linux
Version:
Ubuntu (64-bit)
OK
Cancel



Result

A Type-2 Hypervisor (Oracle VirtualBox) was successfully installed on the host machine, and a Virtual Machine with 3 CPUs, 10 GB RAM, and 10 GB storage was created and configured with a guest operating system, demonstrating hardware virtualization with a higher-resource allocation on a hosted platform.

Experiment 4: Creating and Testing a Snapshot of a Virtual Machine

Aim

To demonstrate virtualization by creating a Snapshot of an existing Virtual Machine (created using a Type-2 Hypervisor) and testing it by restoring the VM to the previous saved version/state.

Software Required

- Oracle VirtualBox (Type-2 Hypervisor) with the source VM from Experiment 3 already created and its guest OS installed
- Sufficient free disk space on the host for storing snapshot delta files

Procedure

1. Start the source virtual machine (e.g., 'SIMATS-VM2') and boot into the guest operating system so it is in a known, working state.
2. In VirtualBox Manager, select the VM, open the 'Snapshots' tab (or Machine → Take Snapshot from the VM window).
3. Click 'Take Snapshot', enter a name (e.g., 'Base-State') and an optional description, then click 'OK'. This captures the current disk state, memory, and configuration of the VM at that point in time.
4. Make a noticeable change inside the guest OS after the snapshot — for example, create a new file, install a small application, or change the desktop background — so that the change can later be used to verify the rollback.
5. Optionally take a second snapshot (e.g., 'After-Change') to represent the modified state, so there are two versions to compare.
6. To test the snapshot, shut down the VM (Machine → Close → Power Off).
7. In the Snapshots tab, right-click the earlier snapshot ('Base-State') and select 'Restore Snapshot' to roll the VM back to that saved version.
8. Confirm the restore when prompted, then start the VM again and log in to the guest OS.
9. Verify that the change made in Step 4 (the new file/application/setting) is no longer present, confirming the VM has been successfully reverted to the previous snapshot's state.
10. This demonstrates the snapshot capability of Type-2 hypervisors, which allows quick save-and-restore of a VM's state without a full reinstall.

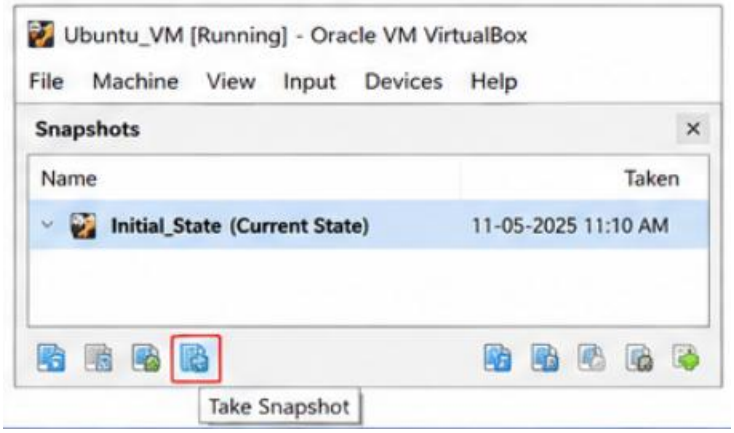
Observation Table

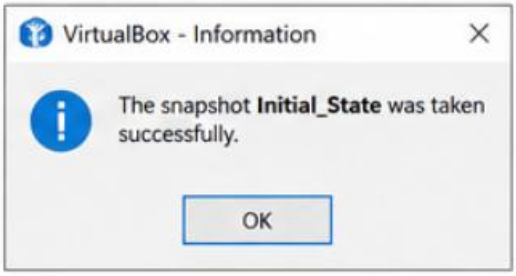
Snapshot	State Captured	Verification After Restore
Base-State	Clean guest OS, before any change	Restored VM does not show the later change
After-Change	Guest OS with the added file/application	Change is present only when this snapshot is active

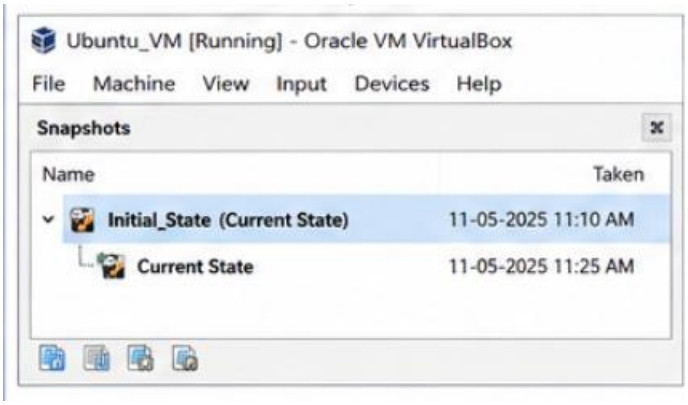
Output Screenshot

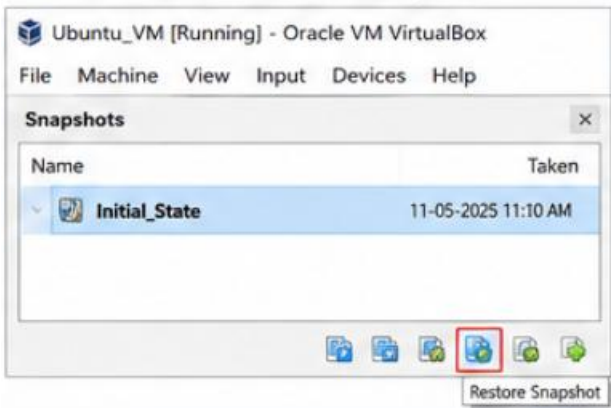

Activities
May 11 11:10
Home
file_v1.txt

Fig 4.1


Ubuntu_VM [Running] - Oracle VM VirtualBox
File Machine View Input Devices Help
Snapshots
Name
Taken
Initial_State (Current State)
11-05-2025 11:10 AM
Take Snapshot


VirtualBox - Information
The snapshot Initial_State was taken successfully.
OK





Result

A snapshot of the existing Virtual Machine was successfully created using the Type-2 Hypervisor's Snapshot feature. The VM was tested by restoring it to the previously saved snapshot, and the guest OS reverted correctly to its earlier state, demonstrating the save-and-restore (rollback) capability of virtualization platforms.